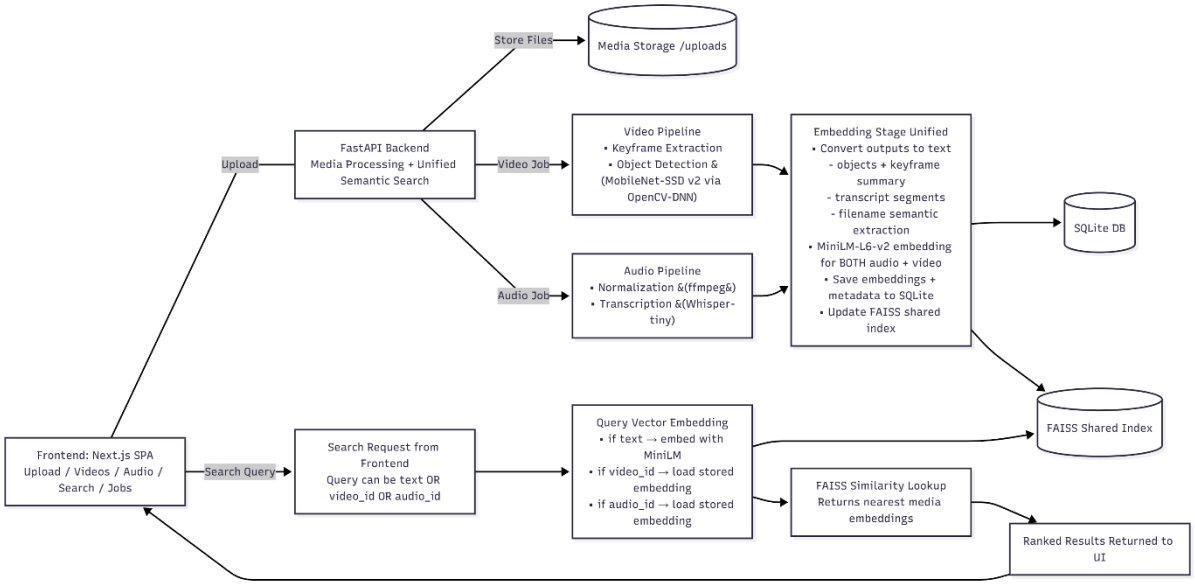

Video Pipeline

1. **Keyframe Extraction**
    a. **Frames read sequentially with OpenCV (cv2.VideoCapture)**
    b. **Histogram comparison between consecutive frames**
    c. **When hist difference > threshold → keyframe stored**
    d. **Reduces redundant frame processing**
2. **Object Detection (MobileNet-SSD v2 via OpenCV-DNN)**
    a. **Each keyframe passed into cv2.dnn.blobFromImage()**
    b. **Forward pass through MobileNet SSD network**
    c. **Outputs bounding boxes + class predictions + confidence**
    d. **Detected objects converted to text summary**
3. **Text + Metadata Semantic Vectorization**
    a. **Filename + detected object labels + summaries converted to text**
    b. **Encoded with MiniLM-L6-v2 → unified embedding space**
    c. **Embeddings stored in SQLite + indexed in FAISS**

Audio Pipeline

1. **Pre-processing**
    a. **Audio normalized and resampled using FFmpeg**
    b. **Ensures consistent input for transcription model**
2. **Transcription with Whisper-tiny**
    a. **Inference runs in chunks/segments**
    b. **Produces text + timestamps + confidence score**
    c. **Long files handled without memory blowout**
3. **Embedding (Unified with Video)**

a. **Transcript text + filename embedded via MiniLM-L6-v2**
b. **Embeddings stored in SQLite + indexed in FAISS**

1. Approach for Implementing Vector Similarity Search Across Different Media Types

The system uses unified text-based embeddings to represent both video and audio content in a shared vector space.

- Video is processed → keyframes extracted → objects recognized → converted to descriptive text
- Audio is transcribed to natural language text
- Filenames are also embedded to contribute semantic context
- All text representations are embedded using MiniLM-L6-v2, producing 384-dimensional vectors that map meaning, not raw pixels or waveforms
- Embeddings are stored in SQLite and indexed in-memory with FAISS L2 similarity search

Because both modalities are embedded using the same model, cross-media semantic search becomes possible.
Searching by a phrase like *"dog barking"* may retrieve videos, audio clips, or both.
Searching by video or audio files (already existing ones) will retrieve their saved embeddings and compare them with other files.

2. How the Solution Handles Large Video + Audio Files

The system is designed so large media files do not block request/response processing:

Aspect	Large Video Files	Large Audio Files
Processing Method	Keyframes extracted rather than full-frame processing	Transcribed in segments, not entire waveform at once
Model Used	MobileNet-SSD operates per-frame (scalable)	Whisper-tiny runs chunk-based for streaming
Execution	Process runs in background worker queue	Same async job queue, buffered workloads
Storage	Only embeddings + metadata are stored in DB	Embeddings stored, raw files do not sit in memory
Indexing	FAISS index loads vectors only, not media files	Lightweight RAM footprint

Video and audio are uploaded once, then processed **asynchronously**, meaning the user isn't blocked — even for long or large inputs. There is however a limitation on the file upload part. Specifically, currently maximum file size is 1GB. To handle more, chunking upload will need to be implemented (backend save chunks and merge).

3. Potential Areas for Improvement With More Compute Resources

Current Approach	Proposed Change	Expected Benefit
MobileNet-SSD	YOLOv8 / DETR	Stronger object detection → richer embeddings
Batch job queue	Celery / Redis / Ray	Better scheduling + distributed scaling
FAISS on CPU	Move FAISS to GPU	Faster retrieval for >1M embeddings
Text-based similarity	Sentence-transformers multimodal model (CLIP)	Enables image similarity without converting to text first

With more hardware, the system could evolve from semantic text-based search into true multimodal embedding reasoning.

4. Trade-offs: Speed vs Accuracy for Video and Audio

Media Type	Faster but Lower Accuracy	Slower but More Accurate
Video	MobileNet-SSD is lightweight and fast; Fewer keyframes → quicker extraction	YOLO / DETR improves recall + complexity; More frames increases accuracy but cost rises
Audio	Whisper-tiny is fast + cheap; Low resource footprint	Whisper-medium/large yield better transcription; Requires GPU + more RAM
Embedding Model	MiniLM fast inference	BERT or large transformer = deeper semantics
Search	FAISS CPU scales quickly	GPU-accelerated FAISS unlocks million-scale index

The system currently favours more on speed and accessibility, while enabling future upgrades if precision becomes the priority.